

ASSIGNMENT – 11.3

2303A51296

Batch-10

Task-1

Prompt: Smart Contact Manager (Arrays & Linked Lists) Contact Manager with add, search, and delete operations using both array (list) and linked list implementations. Include time complexity comparison. Array-Based Implementation and take input from user. Array-Based Implementation

code :

```
class ArrayContactManager:
    def __init__(self):
        self.contacts = []
    def add_contact(self, name, phone):
        self.contacts.append({"name": name,
"phone": phone})
        print(f"Contact '{name}' added.")
    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"].lower() ==
name.lower():
                return contact
        return None
    def delete_contact(self, name):
        for i, contact in enumerate(self.contacts):
            if contact["name"].lower() ==
name.lower():
                self.contacts.pop(i)
                print(f"Contact '{name}' deleted.")
                return
        print(f"Contact '{name}' not found.")
    def display_all(self):
        if not self.contacts:
            print("No contacts found.")
        for c in self.contacts:
            print(f"Name: {c['name']}, Phone:
{c['phone']}")
# Linked List-Based Implementation
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None
```

```

class LinkedListContactManager:
    def __init__(self):
        self.head = None
    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        if not self.head:
            self.head = new_node
        else:
            curr = self.head
            while curr.next:
                curr = curr.next
            curr.next = new_node
        print(f"Contact '{name}' added.")
    def search_contact(self, name):
        curr = self.head
        while curr:
            if curr.name.lower() == name.lower():
                return {"name": curr.name, "phone":
curr.phone}
            curr = curr.next
        return None
    def delete_contact(self, name):
        curr = self.head
        prev = None
        while curr:
            if curr.name.lower() == name.lower():
                if prev:
                    prev.next = curr.next
                else:
                    self.head = curr.next
                print(f"Contact '{name}' deleted.")
                return
            prev = curr
            curr = curr.next
        print(f"Contact '{name}' not found.")
    def display_all(self):
        curr = self.head
        if not curr:
            print("No contacts found.")
        while curr:
            print(f"Name: {curr.name}, Phone:
{curr.phone}")
            curr = curr.next
# ----- Menu Driven Program -----
print("Choose Contact Manager")
print("1. Array Contact Manager")
print("2. Linked List Contact Manager")
choice = int(input("Enter choice: "))

```

```
if choice == 1:
    manager = ArrayContactManager()
else:
    manager = LinkedListContactManager()
while True:
    print("\n1.Add Contact")
    print("2.Search Contact")
    print("3.Delete Contact")
    print("4.Display All Contacts")
    print("5.Exit")
    option = int(input("Enter option: "))
    if option == 1:
        name = input("Enter name: ")
        phone = input("Enter phone: ")
        manager.add_contact(name, phone)
    elif option == 2:
        name = input("Enter name to search: ")
        result = manager.search_contact(name)
        if result:
            print("Found:", result)
        else:
            print("Contact not found")
    elif option == 3:
        name = input("Enter name to delete: ")
        manager.delete_contact(name)
    elif option == 4:
        manager.display_all()
    elif option == 5:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

```

1 #Task 1 - Smart Contact Manager (Arrays & Linked Lists)
2 #Generate a Python Contact Manager with add, search, and delete operations using both array (list) and linked list implementations. Include time complexity comparison.
3 # Array-Based Implementation and take input from user.
4 # Array-Based Implementation
5 class ArrayContactManager:
6     def __init__(self):
7         self.contacts = []
8     def add_contact(self, name, phone):
9         self.contacts.append({"name": name, "phone": phone})
10        print(f"Contact '{name}' added.")
11    def search_contact(self, name):
12        for contact in self.contacts:
13            if contact["name"].lower() == name.lower():

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS

```

PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\.vscode\extensions\ms-python.de
ebugpy\launcher' '98574' '...' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-11.3.py'
Choose Contact Manager
1. Array Contact Manager
2. Linked List Contact Manager
Enter choice: 1

1.Add Contact
2.Search Contact
3.Delete Contact
4.Display All Contacts
5.Exit
Enter option: 1
Enter name: sai
Enter phone: 9121516453
Contact 'sai' added.

1.Add Contact
2.Search Contact
3.Delete Contact
4.Display All Contacts
5.Exit
Enter option: 2
Enter name to search: sai
Found: {'name': 'sai', 'phone': '9121516453'}

1.Add Contact
2.Search Contact
3.Delete Contact
4.Display All Contacts
5.Exit
Enter option: 4
Name: sai, Phone: 9121516453

```

Code Analysis:

- Uses Array (Python list) and Linked List to store contact details (name, phone).
- Supports operations: add, search, delete, and display contacts.
- Array Implementation: Add □ $O(1)$, Search/Delete □ $O(n)$.
- Linked List Implementation: Add □ $O(n)$ (traverse to end), Search/Delete □ $O(n)$.
- Menu-driven program allows the user to manage contacts interactively.

Task-2

Prompt: **Library Book Search System (Queue & Priority Queue)**

Generate a Python Queue for FIFO book requests and a Priority Queue that prioritizes faculty over student requests. Include enqueue and dequeue methods with test cases.

Code :

```
from collections import deque
```

```
import heapq
```

```
class BookRequestQueue:
```

```
    def __init__(self):
```

```
        self.queue = deque()
```

```
    def enqueue(self, requester, book):
```

```

        self.queue.append((requester, book))

        print(f"Request added: {requester} -> '{book}'")

    def dequeue(self):
        if self.queue:
            requester, book = self.queue.popleft()
            print(f"Processing request: {requester} -> '{book}'")
        else:
            print("Queue is empty.")

    def display(self):
        print("Current Queue:", list(self.queue))

# Priority Queue (Faculty = 0, Student = 1)
class PriorityBookQueue:
    def __init__(self):
        self.heap = []
        self.counter = 0

    def enqueue(self, requester, book, role):
        priority = 0 if role.lower() == "faculty" else 1
        heapq.heappush(self.heap, (priority, self.counter, requester, book))
        self.counter += 1
        print(f"[{role.upper()}] Request added: {requester} -> '{book}'")

    def dequeue(self):
        if self.heap:
            priority, _, requester, book = heapq.heappop(self.heap)
            role = "Faculty" if priority == 0 else "Student"
            print(f"Processing [{role}]: {requester} -> '{book}'")
        else:
            print("Priority queue is empty.")

    def display(self):
        print("Priority Queue:", self.heap)

print("Choose Queue Type")

```

```

print("1. Simple Queue")
print("2. Priority Queue")
choice = int(input("Enter choice: "))
if choice == 1:
    q = BookRequestQueue()
    while True:
        print("\n1. Add Request")
        print("2. Process Request")
        print("3. Display Queue")
        print("4. Exit")
        op = int(input("Enter option: "))
        if op == 1:
            name = input("Enter requester name: ")
            book = input("Enter book name: ")
            q.enqueue(name, book)
        elif op == 2:
            q.dequeue()
        elif op == 3:
            q.display()
        elif op == 4:
            break
        else:
            print("Invalid choice")
    else:
        q = PriorityBookQueue()
        while True:
            print("\n1. Add Request")
            print("2. Process Request")
            print("3. Display Queue")
            print("4. Exit")

```

```

op = int(input("Enter option: "))

if op == 1:
    name = input("Enter requester name: ")
    book = input("Enter book name: ")
    role = input("Enter role (faculty/student): ")
    q.enqueue(name, book, role)
elif op == 2:
    q.dequeue()
elif op == 3:
    q.display()
elif op == 4:
    break
else:
    print("Invalid choice")

```

The screenshot shows a VS Code editor with a file explorer on the left containing various files like 'AI ASSISTANT', '1355 AI 8.3.pdf', '2303AS1355_ASSIGN...', 'AI_1355_9.3.pdf', 'AI_1355_10.3.pdf', 'app.log', 'disaster_relief_data.txt', 'disaster.html', 'emp.py', 'employee.py', 'lab-exam-1.py', 'lab-14.py', 'lab-1.py', 'lab-32.py', 'lab-34.py', 'lab-34.py', 'lab-43.py', 'lab-54.py', 'lab-63.py', 'lab-64.py', 'lab-73.py', 'lab-83.py', 'lab-93.py', 'lab-103.py', 'lab-113.py', 'practice.py', 'practice2.py', 'salary.py', and 'smart_contact...'. The main editor window displays a Python script for a queue system. The script defines a class 'BookRequestQueue' with methods 'enqueue', 'dequeue', and 'display'. The terminal output shows the program running and processing requests. The user enters option 1, then option 3, and finally option 4 to exit. The terminal output shows the current queue as ['niraya', 'physics'] after processing the request.

```

114 #Generate a Python Queue for FIFO book requests and a Priority Queue that prioritizes faculty over student requests. Include enqueue and dequeue methods with test cases.
115 from collections import deque
116 import heapq
117 class BookRequestQueue:
118     def __init__(self):
119         self.queue = deque()
120     def enqueue(self, requester, book):
121         self.queue.append((requester, book))

```

Terminal Output:

```

PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\.vscode\extensions\ms-python.debugpy-2025.18.0-wi
4\hond\lib\debugpy\launcher' '53767' '-' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-11-3.py'
Choose Queue Type
1. Simple Queue
2. Priority Queue
Enter choice: 1
1. Add Request
2. Process Request
3. Display Queue
4. Exit
Enter option: 1
Enter requester name: niraya
Enter book name: physics
Request added: niraya -> 'physics'
1. Add Request
2. Process Request
3. Display Queue
4. Exit
Enter option: 3
Current Queue: [('niraya', 'physics')]
1. Add Request
2. Process Request
3. Display Queue
4. Exit
Enter option: 4

```

Code Analysis:

- ☐ Implements FIFO Queue using collections.deque for normal book requests.
- ☐ Implements Priority Queue using heapq where faculty requests have higher priority than students.

- ❑ Main operations: enqueue (add request) and dequeue (process request).
- ❑ Queue Complexity: Enqueue $O(1)$, Dequeue $O(1)$.
- ❑ Priority Queue Complexity: Enqueue $O(\log n)$, Dequeue $O(\log n)$.

Task-3

Prompt: Emergency Help Desk (Stack Implementation)

#Generate a Python Stack for IT Help Desk ticket management using LIFO. Include push, pop, peek, is_empty, and is_full operations. Simulate 5 tickets being raised and resolved.

.

Code :

```
class HelpDeskStack:
    def __init__(self, max_size=10):
        self.stack = []
        self.max_size = max_size
    def push(self, ticket):
        if self.is_full():
            print("Stack is full. Cannot add ticket.")
            return
        self.stack.append(ticket)
        print(f"Ticket raised: '{ticket}'")
    def pop(self):
        if self.is_empty():
            print("No tickets to resolve.")
            return
        ticket = self.stack.pop()
        print(f"Ticket resolved: '{ticket}'")
    def peek(self):
        if self.is_empty():
            print("Stack is empty.")
            return
        print(f"Top ticket: '{self.stack[-1]}'")
```



```

def is_empty(self):
    return len(self.stack) == 0

def is_full(self):
    return len(self.stack) >= self.max_size

def display(self):
    print("Current Tickets (top to bottom):", list(reversed(self.stack)))

size = int(input("Enter maximum stack size: "))
hd = HelpDeskStack(size)

while True:
    print("\n1. Raise Ticket")
    print("2. Resolve Ticket")
    print("3. View Top Ticket")
    print("4. Display Tickets")
    print("5. Check Empty")
    print("6. Check Full")
    print("7. Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
        ticket = input("Enter ticket description: ")
        hd.push(ticket)
    elif choice == 2:
        hd.pop()
    elif choice == 3:
        hd.peek()
    elif choice == 4:
        hd.display()

    elif choice == 5:
        print("Is Empty:", hd.is_empty())
    elif choice == 6:

```

```

        print("Is Full:", hd.is_full())

elif choice == 7:

    print("Exiting...")

    break

else:

    print("Invalid choice")

```

The screenshot shows a VS Code editor with a file explorer on the left containing various files like 'AI ASSISTANT', '1355_AI_8.3.pdf', '2303AI1355_ASSIGN...', 'AI_1355_9.3.pdf', 'AI_1355_10.3.pdf', 'applog', 'disaster_relief_data.txt', 'disaster.html', 'disaster.py', 'emp.py', 'employeee.py', 'lab_exam-1.py', 'lab-1.4.py', 'lab-1.py', 'lab-3.3.py', 'lab-3.4.py', 'lab-3.4.py', 'lab-4.3.py', 'lab-5.4.py', 'lab-6.3.py', 'lab-6.4.py', 'lab-7.3.py', 'lab-8.3.py', 'lab-9.3.py', 'lab-10.3.py', 'lab-11.3.py', 'practice.py', 'practice2.py', 'salary.py', and 'smart_contact...'. The main editor window shows the code for 'lab-11.3.py' which implements a ticket management system using a stack. The terminal output shows the program running and handling user input for raising, resolving, and checking tickets.

```

lab-11.3.py
230     print("2. Resolve Ticket")
231     print("3. View Top Ticket")
232     print("4. Display Tickets")
233     print("5. Check Empty")
234     print("6. Check Full")
235     print("7. Exit")
236     choice = int(input("Enter choice: "))
237     if choice == 1:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```

PS C:\Users\saipr\OneDrive\Desktop\AI assistant> & 'c:\Users\saipr\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\saipr\.vscode\extensions\ms-python.python\python\launcher' '50218' '-.' 'c:\Users\saipr\OneDrive\Desktop\AI assistant\lab-11.3.py'
Enter maximum stack size: 4

1. Raise Ticket
2. Resolve Ticket
3. View Top Ticket
4. Display Tickets
5. Check Empty
6. Check Full
7. Exit
Enter choice: 1
Enter ticket description: wifi not working
Ticket raised: 'wifi not working'

1. Raise Ticket
2. Resolve Ticket
3. View Top Ticket
4. Display Tickets
5. Check Empty
6. Check Full
7. Exit
Enter choice: 2
Ticket resolved: 'wifi not working'

1. Raise Ticket
2. Resolve Ticket
3. View Top Ticket
4. Display Tickets
5. Check Empty
6. Check Full
7. Exit
Enter choice: 4
Current Tickets (top to bottom): []

```

Code Analysis :

- ☐ Uses Stack (LIFO) to manage IT help desk tickets.
- ☐ Operations implemented: push, pop, peek, is_empty, is_full.
- ☐ Push: Adds new ticket to the top of stack.
- ☐ Pop: Resolves the most recent ticket first.
- ☐ Time Complexity: Push $O(1)$, Pop $O(1)$, Peek $O(1)$.

Task-4

Prompt: **Task 4 - Hash Table with Collision Handling (Chaining)**

Generate a Python Hash Table class with insert, search, and delete methods. Use chaining (linked list at each bucket) for collision handling. Add comments explaining each method.

Code :

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(self.size)]
    def _hash(self, key):
        return sum(ord(c) for c in str(key)) % self.size
    def insert(self, key, value):
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                print(f"Updated: '{key}' = '{value}'")
                return
        bucket.append((key, value))
        print(f"Inserted: '{key}' = '{value}'")
    def search(self, key):
        index = self._hash(key)
        bucket = self.table[index]
        for k, v in bucket:
            if k == key:
                print(f"Found: '{key}' = '{v}'")
                return
        print(f"Key '{key}' not found.")
    def delete(self, key):
        index = self._hash(key)
        bucket = self.table[index]
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                print(f"Deleted: '{key}'")
                return
        print(f"Key '{key}' not found.")
    def display(self):
        print("\n--- Hash Table ---")
        for i, bucket in enumerate(self.table):
            if bucket:
                print(f"Bucket {i}: {bucket}")
        print("-----")
size = int(input("Enter hash table size: "))
ht = HashTable(size)
while True:
    print("\n1. Insert")
```

```

print("2. Search")
print("3. Delete")
print("4. Display")
print("5. Exit")
choice = int(input("Enter choice: "))
if choice == 1:
    key = input("Enter key: ")
    value = input("Enter value: ")
    ht.insert(key, value)
elif choice == 2:
    key = input("Enter key to search: ")
    ht.search(key)
elif choice == 3:
    key = input("Enter key to delete: ")
    ht.delete(key)
elif choice == 4:
    ht.display()
elif choice == 5:
    print("Exiting...")
    break
else:
    print("Invalid choice")

```

Output:

```

lab-11.3.py
247     elif choice == 5:
248         print("Is Empty:", hd.is_empty())
249     elif choice == 6:
250         print("Is Full:", hd.is_full())
251     elif choice == 7:
252         print("Exiting...")
253         break
254     else:

```

TERMINAL

```

Enter choice: 1
Enter key: 1
Enter value: 12
Inserted: '1' = '12'

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 2
Enter key to search: 1
Found: '1' = '12'

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 44
Invalid choice

1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter choice: 4

--- Hash Table ---
Bucket 1: [('1', '12')]

```

Code Analysis :

- ❑ Implements a Hash Table using chaining (list at each bucket).
- ❑ Uses a hash function based on ASCII values of the key.
- ❑ Supports insert, search, delete, and display operations.
- ❑ Handles collisions by storing multiple key-value pairs in the same bucket.
- ❑ Average Time Complexity: Insert/Search/Delete $\rightarrow O(1)$, Worst case $O(n)$.

Task-5

Prompt: **Campus Resource Management System**

Campus Resource Management System with: Student Attendance Tracking (Hash Table), Event Registration (Queue), Library Book Borrowing (Priority Queue), Bus Scheduling (Queue), Cafeteria Order Queue (Stack). Implement one feature fully.

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(self.size)]

    def _hash(self, key):
        return sum(ord(c) for c in str(key)) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        bucket = self.table[index]

        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                print(f"Updated: '{key}' = '{value}'")
                return

        bucket.append((key, value))
        print(f"Inserted: '{key}' = '{value}'")

    def search(self, key):
        index = self._hash(key)
        bucket = self.table[index]

        for k, v in bucket:
```

```

        if k == key:
            print(f'Found: '{key}' = '{v}''')
            return
        print(f'Key '{key}' not found.')

def delete(self, key):
    index = self._hash(key)
    bucket = self.table[index]

    for i, (k, v) in enumerate(bucket):
        if k == key:
            bucket.pop(i)
            print(f'Deleted: '{key}''')
            return
    print(f'Key '{key}' not found.')

def display(self):
    print("\n--- Hash Table ---")
    for i, bucket in enumerate(self.table):
        if bucket:
            print(f'Bucket {i}: {bucket}')
    print("-----")

# ---- User Input ----
print("=== Hash Table ===")

size = int(input("Enter hash table size: "))
ht = HashTable(size)

n = int(input("Enter number of elements to insert: "))

for i in range(n):
    key = input("Enter key: ")
    value = input("Enter value: ")
    ht.insert(key, value)

ht.display()

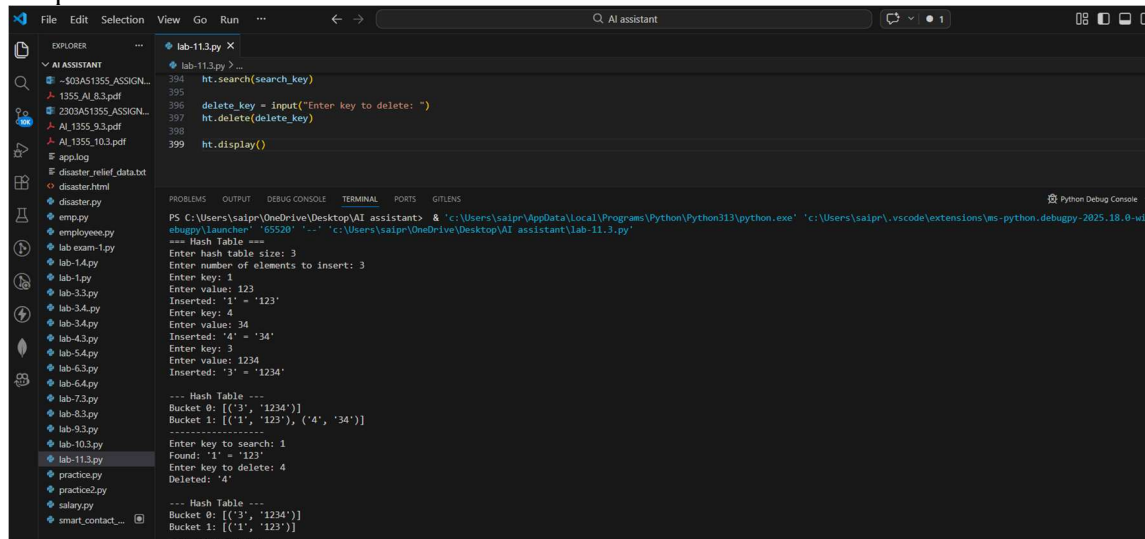
```

```
search_key = input("Enter key to search: ")
ht.search(search_key)
```

```
delete_key = input("Enter key to delete: ")
ht.delete(delete_key)
```

```
ht.display()
```

Output :



```
lab-11.3.py X
394 ht.search(search_key)
395
396 delete_key = input("Enter key to delete: ")
397 ht.delete(delete_key)
398
399 ht.display()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS

PS C:\Users\saipr\OneDrive\Desktop\AI assistant> & 'c:\Users\saipr\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\saipr\.vscode\extensions\ms-python.debugpy-2025.18.0-...
ebugpy\launcher' 65520 '--' 'c:\Users\saipr\OneDrive\Desktop\AI assistant\lab-11.3.py'

```
--- Hash Table ---
Enter hash table size: 3
Enter number of elements to insert: 3
Enter key: 1
Enter value: 123
Inserted: '1' = '123'
Enter key: 4
Enter value: 34
Inserted: '4' = '34'
Enter key: 3
Enter value: 1234
Inserted: '3' = '1234'

--- Hash Table ---
Bucket 0: [('3', '1234')]
Bucket 1: [('1', '123'), ('4', '34')]
-----

Enter key to search: 1
Found: '1' = '123'
Enter key to delete: 4
Deleted: '4'

--- Hash Table ---
Bucket 0: [('3', '1234')]
Bucket 1: [('1', '123')]
-----
```

Code Analysis :

- ☐ Simulates campus systems like attendance tracking, event registration, book borrowing, bus scheduling, and cafeteria orders.
- ☐ Uses multiple data structures: Hash Table, Queue, Priority Queue, and Stack.
- ☐ Fully implements Student Attendance Tracking using a Hash Table.
- ☐ Allows inserting, searching, deleting, and displaying attendance records.
- ☐ Demonstrates efficient data storage and quick retrieval using hashing.